

RAPPORT DE PROJET TUTORÉ

Plateforme de gestion des opportunités de bourses et stages

SongTaaba

Réalisé par : KIMA Ghislain

Filière : Génie Logiciel / Développeur pure

Encadré par : Professeur Christian OUEDRAOGO

Établissement : Université Virtuelle du Burkina Faso

Année académique : 2025 – 2026

Résumé

SongTaaba, qui signifie « l'entraide » en mooré, est une plateforme numérique destinée à faciliter l'accès des étudiants aux opportunités de bourses d'études et de stages. Face à la dispersion des informations sur les bourses disponibles et la difficulté des étudiants à identifier les opportunités adaptées à leur profil et à temps, ce projet propose une solution centralisée combinant scraping automatique de données, gestion intelligente des opportunités et interface mobile intuitive.

La plateforme repose sur une architecture moderne : un backend développé avec **Flask** (Python 3.12), une base de données **PostgreSQL**, une authentification sécurisée par JWT, et un frontend mobile/desktop développé avec Flutter. Le scraping automatique, réalisé avec **BeautifulSoup4**, permet de collecter les offres de bourses et de stages depuis divers sites institutionnels et internationaux.

Ce rapport présente l'ensemble du processus de développement : analyse des besoins, conception, implémentation, tests et perspectives d'évolution.

1. Introduction

1.1 Contexte du projet

Le Burkina Faso, comme de nombreux pays d'Afrique subsaharienne, connaît une croissance significative du nombre d'étudiants cherchant à poursuivre leurs études supérieures, que ce soit sur le plan national ou à l'international. Cependant, l'accès aux opportunités de financement (bourses d'études, allocations de recherche, offres de stages) reste un défi majeur pour la grande majorité de ces étudiants.

Les informations relatives aux bourses sont fragmentées sur de nombreux sites web : ambassades, organisations internationales, ministères, universités étrangères, fondations privées. Cette dispersion oblige les étudiants à consacrer un temps considérable à la recherche manuelle d'informations, avec le risque de manquer des opportunités importantes faute d'une veille systématique.

C'est dans ce contexte que s'inscrit le projet SongTaaba : une plateforme intelligente qui automatise la collecte des offres de bourses et de stages, les centralise et les présente de manière claire et accessible à tous les étudiants, quel que soit leur niveau académique ou leur domaine d'études.

1.2 Objectifs du projet

Les objectifs principaux de SongTaaba sont les suivants :

- **Centraliser les opportunités** : rassembler en un seul endroit toutes les offres de bourses et de stages disponibles pour les étudiants africains.
- **Automatiser la collecte** : mettre en place un système de scraping automatique pour extraire les informations depuis les sites institutionnels et les maintenir à jour.
- **Faciliter la recherche** : offrir des outils de filtrage avancés (pays, niveau d'études, domaine, deadline) pour permettre aux étudiants de trouver rapidement les opportunités correspondant à leur profil.
- **Personnaliser l'expérience** : permettre à chaque étudiant de suivre les bourses qui l'intéressent, de gérer le statut de ses candidatures et de recevoir des alertes sur les nouvelles opportunités.
- **Augmenter le taux de candidatures réussies** : en facilitant l'accès à l'information et en aidant à la préparation des dossiers de candidature.

1.3 Description de la plateforme et des outils utilisés

SongTaaba est une application full-stack composée de plusieurs couches technologiques :

Couche	Technologie	Rôle
Backend	Flask 3.1.1 (Python 3.12)	API REST, logique métier
Base de données	PostgreSQL + SQLAlchemy	Stockage des données
Authentification	Flask-JWT-Extended	Sécurisation des accès
Scraping	BeautifulSoup4 + Requests	Collecte automatique des bourses
Planification	APScheduler	Automatisation du scraping nocturne
Frontend	Flutter (Dart)	Interface mobile et desktop
Migrations	Flask-Migrate (Alembic)	Gestion des versions de BDD

2. Analyse des besoins

2.1 Description des données collectées

Les données traitées par SongTaaba proviennent de deux sources principales :

- **Sources primaires (scraping)** : sites web d'ambassades, d'organisations internationales (Union Africaine, Union Européenne, Agence Française de Développement), de ministères et d'universités étrangères proposant des bourses pour étudiants africains.
- **Sources secondaires** : saisie manuelle par les administrateurs de la plateforme pour les opportunités non disponibles en ligne ou nécessitant une vérification préalable.

2.2 Analyse des utilisateurs cibles et de leurs besoins

SongTaaba cible principalement trois catégories d'utilisateurs :

Profil utilisateur	Besoins principaux
Étudiant en Licence (L1–L3)	Stages académiques, bourses nationales, premier emploi
Étudiant en Master	Bourses internationales, programmes d'excellence, thèses
Étudiant en Doctorat	Bourses de recherche, financement de thèse, mobilité
Administrateur plateforme	Gestion des sites, lancement du scraping, modération

2.3 Étude de marché et concurrence

Plusieurs solutions existent pour aider les étudiants à trouver des bourses, mais aucune ne répond pleinement aux besoins spécifiques des étudiants burkinabè et africains :

Solution	Forces	Limites
Campus France	Bourses françaises bien référencées	Centré sur la France, peu de bourses africaines
Scholarship Portal	Large base internationale	Interface en anglais, non adapté au contexte africain
Groupe Facebook	Communautaire, gratuit	Non structuré, pas de filtres, pas de notifications
SongTaaba	Scraping auto, filtres, suivi candidatures, mobile	En cours de développement

3. Préparation et Exploration des données

3.1 Pré-traitement des données

3.1.1 Nettoyage des données

Les données extraites par scraping nécessitent plusieurs étapes de nettoyage avant leur stockage en base de données :

- **Suppression des espaces superflus** : utilisation de `get_text(strip=True)` de BeautifulSoup pour éliminer les espaces et sauts de ligne parasites.
- **Déduplication** : avant chaque insertion, vérification de l'existence d'une bourse avec le même nom et le même site source.
- **Correction des URLs relatives** : les liens relatifs (commençant par /) sont complétés automatiquement avec le domaine de base du site source.
- **Gestion des valeurs manquantes** : les champs non obligatoires (montant, deadline, description) sont acceptés comme valeurs nulles sans bloquer l'insertion.

3.1.2 Transformation des données

- **Normalisation des montants** : conversion en chaîne de caractères pour uniformiser les formats (euros, FCFA, dollars).
- **Standardisation des dates** : les deadlines sont stockés au format DATE SQL pour permettre le tri et le filtrage.
- **Génération d'UUID** : chaque enregistrement reçoit un identifiant unique généré automatiquement (`uuid.uuid4()`).

4. Conception et Modélisation

4.1 Choix des techniques d'analyse et des modèles

Le projet SongTaaba repose sur des techniques de collecte et de traitement de données web, combinées à une architecture REST pour la diffusion des informations. Les choix techniques sont justifiés comme suit :

- BeautifulSoup4 : choisi pour sa simplicité d'utilisation, sa robustesse face aux structures HTML variées et sa large compatibilité avec les parsers disponibles (lxml, html.parser).
- Requests : bibliothèque HTTP standard Python, choisie pour sa fiabilité et sa gestion complète des en-têtes HTTP (User-Agent, timeout, redirections).
- APScheduler : scheduler léger et intégrable directement dans Flask, sans infrastructure supplémentaire (pas de Celery ou Redis requis).
- PostgreSQL : système de gestion de base de données relationnelle choisi pour sa robustesse, sa gestion des UUIDs et sa compatibilité native avec SQLAlchemy.

4.2 Architecture du système

L'architecture de SongTaaba suit le pattern MVC (Modèle-Vue-Contrôleur) adapté à une API REST :

- Modèles (models/) : définissent la structure des données (Users, Bourses, Sites, Users_Bourses) et les relations entre elles via SQLAlchemy.
- Services (services/) : contiennent la logique métier — opérations CRUD, logique de scraping, gestion des candidatures.
- Routes (routes/) : exposent les endpoints REST, gèrent les requêtes HTTP et délèguent aux services.
- Schemas (schemas/) : assurent la validation des données entrantes et la sérialisation des réponses via Marshmallow.

4.3 Modélisation de la base de données

La base de données comprend quatre tables principales :

Table	Colonnes clés	Rôle
Users	id (UUID), nom, prenom, email, password (hashé), role	Gestion des comptes utilisateurs
Sites	id (UUID), nom, url, sélecteurs CSS, actif, fréquence	Configuration des sites à scraper
Bourses	id (UUID), nom, organisme, pays, niveau, deadline, site_id (FK)	Stockage des opportunités
Users_Bourses	user_id (FK), bourse_id (FK), statut, datetime_interet	Suivi des candidatures (table pivot)

5. Implémentation

5.1 Étapes de développement

Le développement de SongTaaba a suivi une approche itérative organisée en plusieurs phases :

- Phase 1 — Mise en place de l'environnement : configuration de Python 3.12, PostgreSQL, Flask et des extensions. Création de la structure du projet selon le pattern MVC.
- Phase 2 — Modélisation et migrations : définition des modèles SQLAlchemy, génération des migrations Alembic et création des tables en base de données.
- Phase 3 — Développement de l'API REST : implémentation des endpoints CRUD pour chaque ressource, mise en place de l'authentification JWT.
- Phase 4 — Développement du scraper : implémentation du module de scraping avec BeautifulSoup4, gestion des sélecteurs CSS configurables, déduplication et sauvegarde en base.
- Phase 5 — Automatisation : intégration d'APScheduler pour le lancement automatique du scraping chaque nuit à 02h00.
- Phase 6 — Frontend Flutter : développement de l'interface mobile/desktop connectée à l'API Flask.

5.2 Modules et composants clés

5.2.1 Module de scraping (services/scraping.py)

Le cœur du système de collecte repose sur la fonction `scraper_site(site)` qui :

- Télécharge le HTML de la page via `requests.get()` avec un User-Agent réaliste pour éviter les blocages.
- Parse le contenu avec BeautifulSoup en utilisant le parser lxml.
- Applique les sélecteurs CSS configurés dans le modèle Site pour extraire les titres, descriptions, deadlines et liens.
- Vérifie l'absence de doublons avant chaque insertion.
- Met à jour le champ `derniere_execution` du site après chaque scraping réussi.

5.2.2 Module d'authentification

L'authentification repose sur Flask-JWT-Extended. À la connexion, un token JWT est généré et retourné au client. Ce token est ensuite inclus dans le header `Authorization: Bearer <token>` de chaque requête protégée. Les mots de passe sont hachés avec Werkzeug (`generate_password_hash`) et jamais stockés en clair.

5.2.3 Interface Flutter

Le frontend Flutter communique avec l'API via le package `http`. Il comprend : une page de login/inscription, une page d'accueil avec liste des bourses filtrables, une page de favoris et

une page de notifications. La navigation est gérée par une `BottomNavigationBar` et les détails s'affichent via des `ModalBottomSheets`.

6. Résultats de l'analyse

6.1 Résultats et interprétation

À ce stade du développement, les premiers résultats obtenus sont les suivants :

- **API REST fonctionnelle** : tous les endpoints définis (authentification, CRUD bourses, sites, candidatures) répondent correctement aux requêtes HTTP.
- **Scraping implémenté mais pas complètement opérationnel** : le module de scraping est capable d'extraire des données depuis des sites web en utilisant des sélecteurs CSS configurables, avec déduplication automatique.
- **Interface Flutter connectée** : le frontend communique avec l'API, permettant l'inscription, la connexion, la consultation des bourses et le marquage d'intérêts.
- **Automatisation planifiée** : APScheduler est intégré et programmé pour lancer le scraping chaque nuit à 02h00, sans intervention manuelle.
-

7. Tests et Validation

La campagne de tests n'a pas encore été effectuée. Elle est planifiée pour une phase ultérieure du projet.

8. Performances et Optimisation

8.1 Évaluation des performances

Performances non évaluées. Les tests de performance n'ont pas encore été effectués. Ils sont programmés pour une phase ultérieure du projet.

8.2 Optimisations mises en place

Optimisation en attente. L'optimisation du code et des ressources sera réalisée après la validation fonctionnelle.

9. Sécurité et protection des données

9.1 Mesures de sécurité

- Hachage des mots de passe : Werkzeug (generate_password_hash / check_password_hash) utilise l'algorithme PBKDF2-SHA256 avec salage automatique. Aucun mot de passe n'est stocké en clair.
- Authentification JWT : chaque requête protégée requiert un token JWT valide, transmis dans le header HTTP Authorization. Le token a une durée de vie limitée.
- Variables d'environnement : les clés secrètes (SECRET_KEY, JWT_SECRET_KEY, DATABASE_URL) sont stockées dans un fichier .env non versionné, chargé via python-dotenv.
- CORS configuré : Flask-CORS contrôle les origines autorisées à accéder à l'API.
- Validation des entrées : Marshmallow valide toutes les données entrantes avant tout traitement, prévenant les injections et les données malformées.

9.2 Gestion des données personnelles

SongTaaba collecte uniquement les données strictement nécessaires au fonctionnement de la plateforme : nom, prénom, adresse email et mot de passe hashé. Aucune donnée sensible supplémentaire n'est collectée.

Les données des utilisateurs peuvent être supprimées à leur demande via l'endpoint DELETE /api/users/<id>. Le projet est conçu dans le respect des principes de minimisation des données et de transparence, conformément aux bonnes pratiques de protection des données personnelles.

10. Conclusion

10.1 Résumé des réalisations

Le projet SongTaaba a permis de développer une plateforme fonctionnelle de gestion des opportunités de bourses et de stages, répondant aux besoins identifiés des étudiants burkinabè et africains. Les réalisations principales sont :

- Une API REST complète développée avec Flask, couvrant l'authentification, la gestion des utilisateurs, des bourses, des sites de scraping et des candidatures.
- Un système de scraping automatique basé sur BeautifulSoup4, configurable via des sélecteurs CSS et planifié chaque nuit avec APScheduler.
- Un frontend mobile et desktop développé avec Flutter, connecté à l'API et offrant une interface intuitive et moderne.
- Une base de données PostgreSQL bien structurée, avec migrations gérées par Alembic, garantissant l'intégrité et l'évolutivité des données.

10.2 Perspectives d'amélioration

- Scraping Selenium : pour les sites dynamiques (JavaScript), remplacer ou compléter Requests/BeautifulSoup par Selenium ou Playwright.
- Envisager une collaboration directe à travers avec les structures boursières.
- Notifications push : intégrer Firebase Cloud Messaging (FCM) pour envoyer des notifications push aux utilisateurs sur mobile.
- Moteur de recommandation : développer un algorithme de recommandation personnalisée basé sur le profil de l'étudiant (niveau, domaine, pays cible).
- Assistance à la candidature : ajouter un module d'aide à la rédaction de lettres de motivation et de CV adapté au contexte des bourses.

10.3 Développement futur

- Une plateforme multi-pays couvrant l'ensemble de l'Afrique de l'Ouest, avec des partenariats institutionnels.
- Une version web progressive (PWA) en complément de l'application mobile Flutter.
- Un tableau de bord analytique pour les institutions souhaitant suivre l'intérêt des étudiants pour leurs programmes.
- Une intégration avec des plateformes de candidature en ligne pour permettre la soumission directe des dossiers depuis SongTaaba.

11. Annexes

Annexe A — Structure du projet

```
songtaaba/
├── README.MD
├── backend/
│   ├── app.py                # Factory Flask
│   ├── config.py             # Configurations (dev/prod/test)
│   ├── extensions.py         # db, ma, jwt, migrate
│   ├── requirements.txt
│   ├── models/
│   │   ├── user.py           # Modèle utilisateurs
│   │   ├── site.py           # Modèle sites de scraping
│   │   ├── bourse.py         # Modèle bourses & stages
│   │   └── users_bourses.py  # Table pivot intérêts/candidatures
│   ├── schemas/
│   │   ├── user_schema.py
│   │   ├── site_schema.py
│   │   ├── bourse_schema.py
│   │   └── users_bourses_schema.py
│   ├── services/
│   │   ├── user_service.py
│   │   ├── bourse_service.py
│   │   ├── site_service.py
│   │   ├── users_bourses_service.py
│   │   └── scraping.py       # Scraping BeautifulSoup
│   ├── resources/
│   │   ├── user_resource.py
│   │   ├── bourse_resource.py
│   │   ├── site_resource.py
│   │   └── users_bourses_resource.py
│   ├── routes/
│   │   ├── auth_routes.py
│   │   ├── user_routes.py
│   │   ├── bourse_routes.py
│   │   ├── site_routes.py
│   │   └── users_bourses_routes.py
│   ├── migrations/          # Alembic migrations
│   └── tests/
│       ├── test_auth.py
│       └── test_user.py
├── frontend/
│   ├── lib/
│   │   └── main.dart         # Point d'entrée Flutter
│   └── android/
```

```

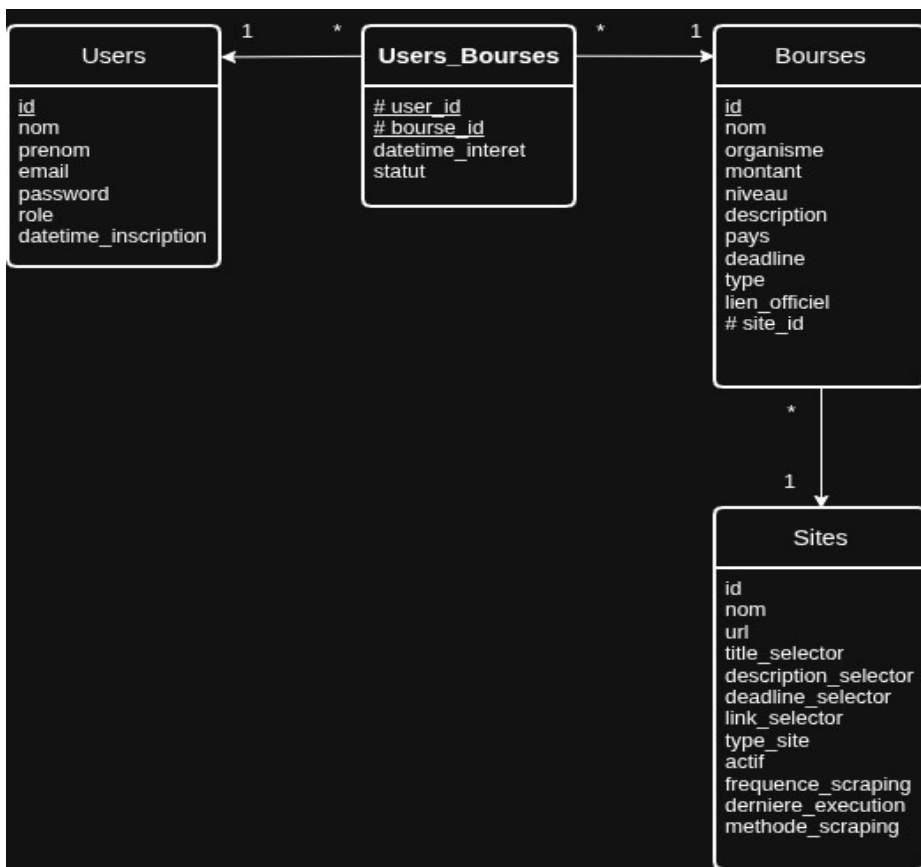
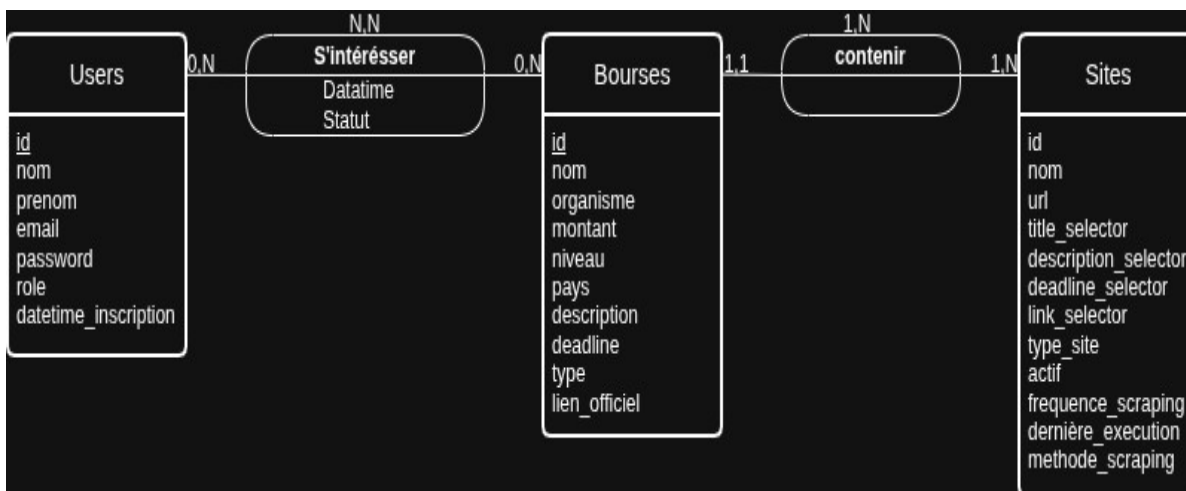
├── ios/
├── linux/
├── macos/
├── web/
├── windows/
├── pubspec.yaml
├── docs/
│   ├── database/
│   │   ├── mcd.png          # Modèle Conceptuel
│   │   ├── mld.png          # Modèle Logique
│   │   └── mpd.png          # Modèle Physique
│   └── ui/
│       └── interface.drawio  # Maquettes interfaces

```

Annexe B — Principaux endpoints de l'API

Méthode	Endpoint	Auth	Description
POST	/api/users/register	Non	Créer un compte
POST	/api/auth/login	Non	Se connecter
GET	/api/auth/me	JWT	Profil connecté
GET	/api/bourses/	Non	Liste des bourses
GET	/api/bourses/?pays=France	Non	Filtrer par pays
POST	/api/bourses/	JWT	Créer une bourse
POST	/api/sites/scrape	JWT	Lancer le scraping de tous les sites
POST	/api/users-bourses/	JWT	Marquer un intérêt
PATCH	/api/users-bourses/{uid}/{bid}	JWT	Changer le statut de candidature

Annexe C : Modélisation de la Base de données



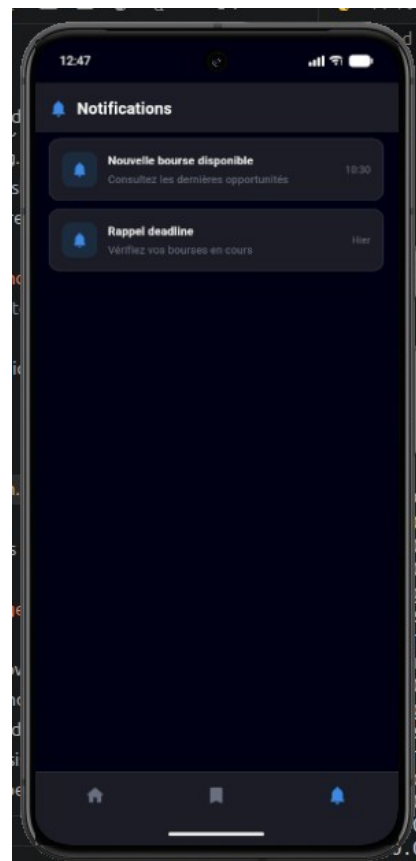
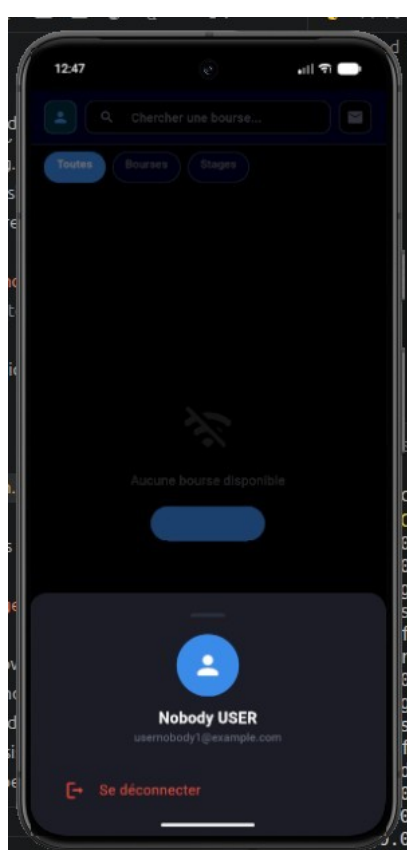
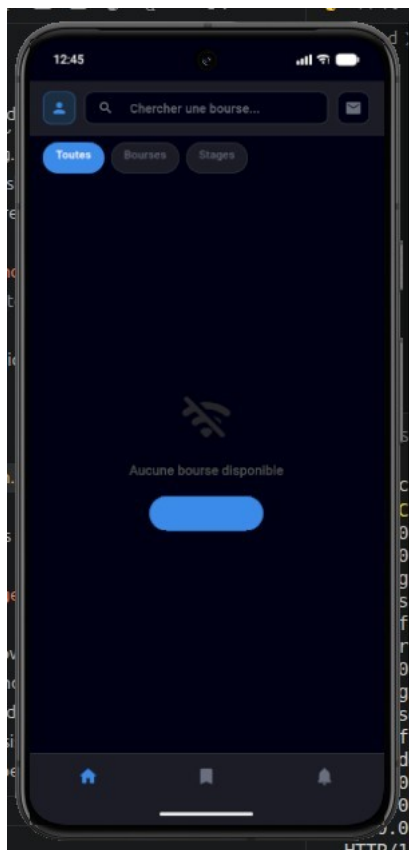
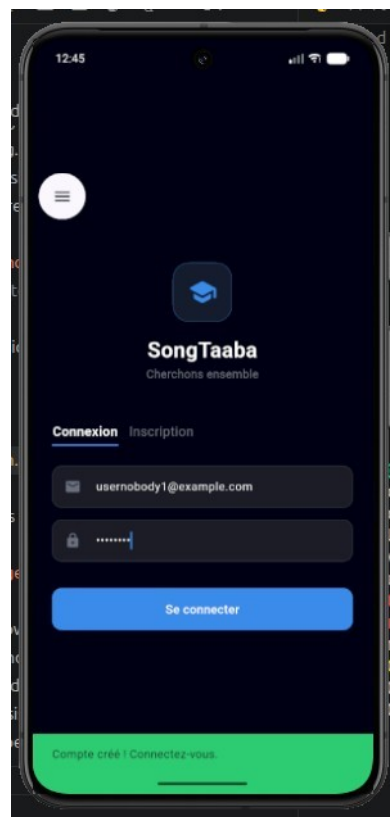
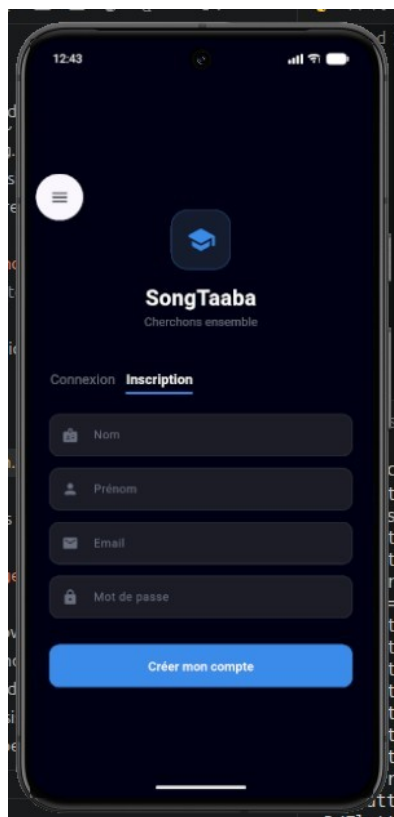
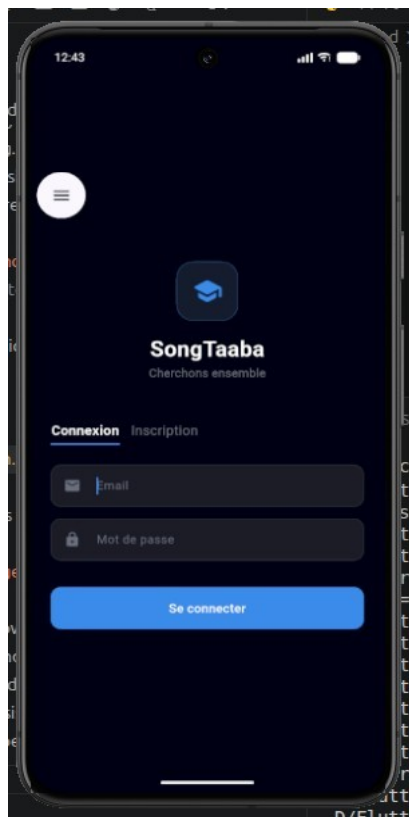
Users				
Nom	Type	Taille	Contrainte	Référence
id	UUID		UNIQUE, NOT NULL, PK	
nom	VARCHAR	50	NOT NULL	
prenom	VARCHAR	100	NOT NULL	
email	VARCHAR	150	UNIQUE, NOT NULL	
password	VARCHAR	255	NOT NULL	
role	VARCHAR	50	DEFAULT 'user'	
datetime_inscription	DATETIME		DEFAULT CURRENT_TIMESTAMP	

Bourses				
Nom	Type	Taille	Contrainte	Référence
id	UUID		UNIQUE, NOT NULL, PK	
nom	VARCHAR	255	NOT NULL	
organisme	VARCHAR	255		
montant	DECIMAL	10,2		
niveau	VARCHAR	100		
description	TEXT			
pays	VARCHAR	100		
deadline	DATE			
type	VARCHAR	100		
lien_officiel	VARCHAR	255		
site_id	UUID		FK, NOT NULL	Sites(id)

Sites				
Nom	Type	Taille	Contrainte	Référence
id	UUID		UNIQUE, NOT NULL, PK	
nom	VARCHAR	255	NOT NULL	
url	VARCHAR	255	NOT NULL	
title_selector	VARCHAR	255		
description_selector	VARCHAR	255		
deadline_selector	VARCHAR	255		
link_selector	VARCHAR	255		
type_site	VARCHAR	100		
actif	BOOLEAN		DEFAULT TRUE	
frequence_scraping	INT		(jours)	
derniere_execution	DATETIME			
methode_scraping	VARCHAR	50		

Users_Bourses				
Nom	Type	Taille	Contrainte	Référence
user_id	UUID		PK (composite), FK	Users(id)
bourse_id	UUID		PK (composite), FK	Bourses(id)
datetime_interet	DATETIME		DEFAULT CURRENT_TIMESTAMP	
statut	VARCHAR	50	DEFAULT 'interessé'	

Annexe D : Démonstration visuelle



Annexe E : Lien GitHub

<https://github.com/Ghislain-KIMA/SongTaaba.git>